Projeto RotaFácil — Viagens

Guia do trabalho · Fase 1 (baseada na Aula 1 do TaskFlow)

Tipos, Interfaces e renderização de um Array de Objetos em TypeScript

Prévia da tela final

Assim deve ficar o projeto rodando no navegador (`npm run dev`):

Tela inicial do RotaFácil: nome e versão no topo, painel com totais, aviso da próxima viagem a planejar e a lista com as cores por situação.

1. O que o trabalho pede

Com base no projeto **TaskFlow** desenvolvido em aula, cada grupo deve construir um sistema próprio com um tema diferente. O tema do nosso grupo é **viagens**. Nesta primeira fase é preciso entregar:

- **Tela inicial** com o **nome** e a **versão** do sistema (e a data de hoje);
- Os arquivos **index.html**, **style.css**, **dom.ts** e **main.ts**;
- Uso de **Types** e **Interface** para modelar o registro principal (**tipos.ts**);
- Um **Array de Objetos** conforme o tema, renderizado na tela (**dados.ts**);
- Realizar o **versionamento** desta primeira fase do projeto.

2. Como adaptamos o TaskFlow para o tema Viagens

A estrutura é a mesma da Aula 1; só mudam os nomes e os dados:

Elemento no TaskFlow	No nosso projeto (RotaFácil)
Registro principal: Tarefa	Viagem (interface em tipos.ts)
prioridade (define a cor)	situacao: planejada / confirmada / concluida

status	tipo: lazer / trabalho / familia
tags (lista)	atividades (lista de pontos turísticos)
título	destino + país

A cor da barra lateral vem da folha de estilo: **planejada = laranja**, **confirmada = verde**, **concluida = cinza**. O código só escolhe a classe (`viagem--${viagem.situacao}`) — a cor sai certa sozinha.

3. Estrutura dos arquivos

```
viagens/
■■■ index.html          <- tela inicial (nome, versao e data)
■■■ package.json        <- scripts: dev e verificar
■■■ tsconfig.json       <- configuracao do TypeScript (strict)
■■■ css/
■   ■■■ style.css       <- cores por situacao + painel
■■■ src/
    ■■■ dom.ts          <- vem pronto (funcao "elemento")
    ■■■ tipos.ts        <- TYPES + INTERFACE Viagem
    ■■■ dados.ts        <- ARRAY de objetos (6 viagens)
    ■■■ main.ts         <- renderizacao com os 3 lacos
```

4. O código explicado (as partes que valem nota)

4.1 — tipos.ts: Types e Interface

O **type** (union type) fecha os valores possíveis de um campo. A **interface** é o contrato: todo objeto Viagem precisa ter essas propriedades, e o compilador confere antes da página rodar.

```
export type Situacao = "planejada" | "confirmada" | "concluida";
export type TipoViagem = "lazer" | "trabalho" | "familia";

export interface Viagem {
  readonly id: number;    // readonly: nao muda depois de criada
  destino: string;
  pais: string;
  situacao: Situacao;     // so aceita valores do union type
  tipo: TipoViagem;
  observacao?: string;    // o ? indica propriedade OPCIONAL
  atividades: string[];   // array de textos (faz o papel das tags)
  criadaEm: Date;
}
```

4.2 — dados.ts: o Array de Objetos

`Viagem[]` significa “array de Viagem”: só aceita itens desse tipo. Os itens são acessados por índice, que começa em zero.

```
import type { Viagem } from "./tipos";

export const viagens: Viagem[] = [
  { id: 1, destino: "Paris", pais: "Franca", situacao: "confirmada",
    tipo: "lazer", atividades: ["Torre Eiffel","Louvre"], criadaEm: new Date() },
  { id: 2, destino: "Lisboa", pais: "Portugal", situacao: "concluida",
    tipo: "familia", atividades: ["Belem","Sintra"], criadaEm: new Date() },
  // ... (6 viagens no total)
];
```

4.3 — main.ts: os 3 laços que desenham a tela

1. **for...of** — percorre todas as viagens e cria um <li> para cada uma (createElement + className + append);
2. **for com índice** — conta quantas viagens ainda estão *planejadas* e preenche o painel;
3. **while** — procura a primeira viagem planejada e mostra o aviso “Comece a planejar”.

5. O que deve ser feito — passo a passo

Siga na ordem. O projeto só funciona pelo servidor do Vite (não abra o HTML com dois cliques).

```
# 1) Instalar as dependencias (primeira vez)
cd viagens
npm install

# 2) Rodar o projeto em modo desenvolvimento
npm run dev
# Abra o endereco que aparecer: http://localhost:5173

# 3) Conferir os tipos (TypeScript)
npm run verificar      # roda:  tsc --noEmit  -> deve passar sem erros
```

■ **Atenção:** não abra o `index.html` com dois cliques. Pelo endereço file:// os módulos não carregam e a página aparece em branco. Use sempre o `npm run dev`.

6. Versionamento da primeira fase (Git)

Para "realizar o versionamento" pedido no enunciado, inicialize um repositório Git e faça o primeiro commit desta fase. Crie antes um `.gitignore` para não enviar a pasta node_modules.

```
# .gitignore  (crie este arquivo na raiz do projeto)
node_modules
dist

# No terminal, dentro da pasta viagens:
git init
git add .
git commit -m "Fase 1: tela inicial, tipos, interface e lista de viagens"

# (opcional) enviar para o GitHub:
git branch -M main
git remote add origin <URL-do-repositorio>
git push -u origin main
```

7. Checklist de entrega

- ■ Tela inicial mostra o NOME (RotaFácil) e a VERSÃO (v0.1.0) + data;
- ■ index.html, style.css, dom.ts e main.ts presentes e funcionando;
- ■ tipos.ts com Types (union) e Interface Viagem;
- ■ dados.ts com o Array de Objetos (lista de viagens) sendo renderizado na tela;
- ■ As 6 viagens aparecem com as cores por situação e o painel com os totais;
- ■ npm run verificar passa sem erros de tipo;
- ■ Versionamento feito com Git (git init + primeiro commit).